Program and Branch Name: *BTech CSE*

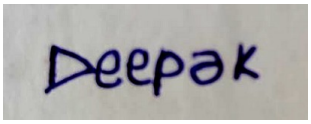
PBL Progress Report – II

Field	Details
Course Name	Data Structures & Algorithms – II
Course Code	CCSE0301
Faculty Name	Mr. Pankaj Rajput
Assignment Type	Individual Assignment
Student Name	Deepak Joshi
ERP ID / Roll No.	2501331720045
Team/Group No.	N/A – Individual Assignment
Problem Statement / Project Title	INTELLIGENT STUDENT MENTORSHIP ALLOCATION SYSTEM
SDG(s)	SDG 4 – Quality Education
Reporting Month	Month 2
Date of Submission	21-10-2026

Student Declaration

I certify that this assignment is my own work, with all sources appropriately acknowledged. I understand that a 0% mark and further academic penalties may apply if the content is found to be plagiarised or inappropriately AI-generated.

Student Name & Signature:

Deepak Joshi


Date: [21-10-2026]

PBL Progress Report – II

Review 2: Design and Implementation Progress

Field	Details
1. Problem Statement / Assignment or Project Title	Intelligent Student Mentorship Allocation System – matches mentees with faculty or senior-student mentors based on subject need, interests, availability and mentor workload.
2. Progress Since Review 1	Review 1 covered the problem definition, entities, workflow and literature review. This month I designed the database, defined how a mentor is scored for a mentee, and built a first working version of the allocation, which I tested on a small sample dataset.
3. Data Storage (SQL Database)	• Data is stored in an SQL database with four tables: Student, Mentor, MentorshipRequest and Allocation. • Student holds interests, weak subjects, free time slots and preferred mentor type. Mentor holds expertise, free time slots, maximum capacity and current load. • Allocation stores the final mentor–mentee pairs with their match score, so results are saved permanently.
4. Working Data Structures (Arrays)	• Records are fetched from the database with SQL queries and loaded into arrays (one for mentees, one for mentors) so the matching can run in memory. • A score array holds the fit scores of each mentor for the mentee being processed. • After allocation, the results and updated mentor load values are written back to the database.
5. How Mentor Fit Is Scored	Each mentor gets a score out of 100 for a given mentee: • Subject match (40): (weak subjects covered by the mentor's expertise ÷ total weak subjects) × 40. • Interest overlap (25): (common interests ÷ mentee's interests) × 25. • Availability (20): (common free time slots ÷ mentee's free slots) × 20, capped at 20. Zero common slots means the mentor is rejected. • Workload balance (15): (remaining capacity ÷ maximum capacity) × 15, so less loaded mentors score higher. • A mentor is only considered if the total is at least 50 and he or she still has capacity.
6. Allocation Algorithm (Greedy + Graph)	• Searching: a linear search over the mentor array removes mentors who are at full capacity or share no free time slot with the mentee. Binary search is used on the sorted array for quick lookup by mentor ID. • Sorting: remaining mentors are sorted by score in descending order (merge sort / heap sort). Mentees are also sorted so those with the most weak subjects are handled first. • Greedy approach: for each mentee, the highest-scoring mentor with free capacity and a score above the minimum threshold is chosen, and that mentor's load is updated immediately. The workload part of the score makes already busy mentors less

	attractive, which keeps the load balanced. • Graph: mentees and mentors are nodes in a bipartite graph stored as an adjacency list. An edge exists when the score is above the threshold. Mentees with no edges are flagged for the coordinator, and BFS is used to look for mentors through related subjects. • Priority queue (max-heap) can be used to pull the best mentor for each mentee without fully sorting the list.
7. Work Completed During Month 2	• Designed the database tables and wrote the SQL queries for inserting and fetching data. • Prepared a sample dataset of mentors and mentees. • Implemented loading data into arrays, the scoring function, searching and sorting steps. • Implemented the greedy allocation with capacity checks and the write-back to the database. • Tested on the sample data and adjusted the weights after a few poor matches.
8. Evidence / Supporting Material	Database schema, sample dataset, scoring table, prototype code and sample allocation output with test results.
9. Challenges Faced	• Deciding the score weights so that subject need matters most without ignoring availability. • Greedy allocation does not always give the best overall result, because early mentees can take the best mentors. Sorting mentees by need first reduces this. • Handling mentees whose preferred mentor is already full. • Keeping mentor load balanced while still giving good matches.
10. Individual Contribution	As this is an individual assignment, all design, coding, testing and documentation was done by Deepak Joshi.
11. Overall Progress (%)	75% – database design, scoring logic, greedy allocation and first round of testing are done. Final optimisation, more testing and the report are left.
12. Plan for Next Review	• Test on a larger dataset and compare running time and balance with a simple manual or random allocation. • Improve handling of full mentors, possibly with backtracking to reassign a previous mentee. • Write the time and space complexity of each step. • Fix remaining edge cases and prepare the final report and demo.